- *Augmentation layer*—The "A" in RAG is the augmentation layer. This layer combines the most relevant retrieved contexts with the original user query, carefully preparing an enhanced prompt that provides the language model with comprehensive background information while ensuring everything fits within the context window.
- *Generation engine*—The "G" in RAG is the generation engine. This engine uses LLMs to transform the augmented context and query into coherent responses. This engine uses the intricacies of the targeted LLM to maintain high relevance, accuracy, and coherence with the original information sources.

While these components form the core of this approach, there are several ongoing research directions and variations of RAG systems that aim to improve their capabilities.

One variation is *MultiHop-RAG,* where the system recursively retrieves additional relevant contexts based on the initial results and reinforcement learning–based retrieval. The latter is used to learn to optimize the retrieval process for specific task objectives that require reasoning over multiple pieces of supporting evidence.

Another variation is *hypothetical document embedding* (HyDE). The idea is to go beyond simply indexing and retrieving real-world documents. Instead, the system could generate synthetic document representations that do not correspond to any existing text but are designed to complement and enhance the information retrieval process across various tasks (e.g. web search, Q/A, and so on).

For the sake of simplicity, we will focus on implementing this core RAG with a grounding system. The key components will include document indexing, a retrieval mechanism, a grounding layer to identify noticeable facts and relationships, and an augmented generation process that incorporates the grounded information.

In chapter 11, we will explore a more realistic and complex example that uses RAG for a real-world application scenario. This will allow us to dive deeper into the nuances and advanced techniques within the RAG paradigm, building upon the foundational knowledge established in this chapter.

7.2.3 Technical architecture overview

Similar to the summarization web application, the RAG service utilizes relevant libraries and frameworks. The key difference is the inclusion of additional procedures to enable the RAG workflow. Here is the list of technologies that we are using:

- *Frontend*—ReAct
- *Backend*—Next.js
- *Vector database*—HNSWLib
- *Embedding model*—GoogleAI
- *LLM integration*—GoogleAI
- *Grounding*—Google grounding AI

Grounding is basically a form of verification check. It helps verify the factual accuracy of the generated responses by cross-checking them against a corpus of reliable